ENGINEERING ESSENTIALS FAANG INTERVIEW MASTER SERIES

Deep Learning

Neurons, backprop, optimizers, CNNs and RNNs — how machines learn representations from raw data.

Neural Nets | Backprop | Activations | Optimizers | CNN | RNN / LSTM | Batch Norm | Dropout

VOLUME 20
THE DEEP LEARNING ISSUE

COMPREHENSIVE
DEEP DIVE + REVISION

LEVEL
SWE → STAFF

IN THIS ISSUE

Contents

SECTION 01

01 Overview — What It Is

Deep Learning is a subfield of machine learning that uses **multi-layered artificial neural networks** to learn hierarchical representations directly from raw data. Instead of handcrafting features, the network learns them automatically through a training process that adjusts millions (or billions) of parameters.

Key idea: Stack many parametric transformations (layers), apply non-linear activation functions between them, define a loss measuring prediction quality, and iteratively minimize the loss via gradient descent by flowing gradients backward through every layer.

"Deep" = more than one hidden layer. Depth enables the network to compose simple features into complex ones: edges → textures → parts → objects (in vision); characters → morphemes → words → meaning (in NLP).

SECTION 02

02 Why It Exists

Era	Limitation	Deep Learning Solution
Pre-1980s	Linear models can't capture non-linear patterns	Non-linear activations + multiple layers
1980s–2000s	Shallow nets, vanishing gradients, limited data	Better init, ReLU, more data
2006–2012	Hand-engineered features dominate	Pre-training, GPUs, AlexNet (2012) breakthrough
2012–present	Dataset scale + compute	Large-scale training on GPUs/TPUs, transformers

Deep Learning exists because: 1. **Raw data is high-dimensional** — pixels, tokens, audio samples. Hand-engineered features are brittle. 2. **Universal approximation theorem** — an MLP with one hidden layer can approximate any continuous function. Depth makes this efficient. 3. **GPU parallelism** — matrix multiplications are embarrassingly parallel. 4. **Data availability** — ImageNet (1.2M images), web text corpora, etc.

SECTION 03

03 Why FAANG Cares (Be Specific)

- **Google/DeepMind** — Search ranking, Google Translate (Transformer), AlphaFold, Gemini. Every ranking model at Google is a deep neural net.
- **Meta/Facebook** — Recommendation systems (DLRM), content moderation (image/video classifiers), ads CTR models, Llama LLMs.
- **Amazon** — Alexa (RNN/Transformer ASR), product recommendation, fraud detection, Alexa voice embeddings.
- **Apple** — Face ID (CNN embeddings), Siri (seq2seq), on-device ML (CoreML), photo classification.
- **Netflix** — Recommendation via deep collaborative filtering, thumbnail A/B test click-through CNNs.
- **OpenAI/Microsoft** — GPT models, Copilot, Azure ML infrastructure.

Why interviewers probe this deeply: - Deep learning engineers need to debug training (exploding/vanishing gradients, overfitting, dead neurons). - Architectural choices (CNN vs RNN vs Transformer) have large cost/quality trade-offs at FAANG scale. - Knowing backprop from first principles = knowing how to debug gradient flow issues. - Knowing normalization + initialization = avoiding 90% of training instability issues.

SECTION 04

04 Core Concepts

The Perceptron — Atomic Unit

The **perceptron** (Rosenblatt, 1958) is a single artificial neuron:

DIAGRAM

```
y = f( w1*x1 + w2*x2 + ... + wn*xn + b )
  = f( W·x + b )
```

- **x** = input vector
- **W** = weight vector (what the network learns)
- **b** = bias scalar (shifts the decision boundary)
- **f** = activation function (adds non-linearity)
- **y** = output

A single perceptron is a linear classifier — it can only separate linearly separable classes. Famous failure: XOR problem (fixed by adding hidden layers).

Interview takeaway: A perceptron without a non-linear activation is just linear regression. Non-linearity is what makes neural nets powerful.

Multilayer Perceptron (MLP)

Stack multiple perceptrons in layers:

DIAGRAM

```
Input Layer → Hidden Layer 1 → Hidden Layer 2 → ... → Output Layer
```

Each layer: `h = f(W·h_prev + b)`

Why layers matter: Each layer learns a progressively more abstract representation: - Layer 1 might detect low-level features (edges, n-gram frequencies) - Layer 2 combines those into mid-level patterns - Layer N learns task-specific abstractions

Parameters: `sum over layers of (n_in * n_out + n_out)` — weights + biases per layer.

Activation Functions

Activation functions introduce **non-linearity** — without them, stacking layers collapses to a single linear transformation (`W3 * W2 * W1 * x = W_combined * x`).

Sigmoid

DIAGRAM

```
σ(x) = 1 / (1 + e^(-x))    range: (0, 1)
```

- Outputs are probability-like (range 0–1)
- **Problem:** Saturates at extremes → gradients ≈ 0 → **vanishing gradient**
- **Problem:** Outputs not zero-centered → zig-zag gradient updates
- **Use:** Binary output layer only (never in hidden layers today)

Tanh

DIAGRAM

```
tanh(x) = (e^x - e^(-x)) / (e^x + e^(-x))    range: (-1, 1)
```

- Zero-centered (better than sigmoid for hidden layers)
- Still saturates → vanishing gradients
- **Use:** RNN hidden states (historically common)

ReLU (Rectified Linear Unit)

DIAGRAM

```
ReLU(x) = max(0, x)
```

- **No saturation for x > 0** → gradients flow freely
- Computationally cheap (just a threshold)
- **Problem: Dead neurons** — if a neuron's input is always negative, it never activates, gradient = 0 forever, it "dies"
- **Use:** Default for hidden layers in most modern networks

Leaky ReLU

DIAGRAM

```
LeakyReLU(x) = max(0.01*x, x)
```

- Small negative slope (0.01) prevents dead neurons
- Small computational overhead vs ReLU
- **Use:** When dead neuron problem is observed

GELU (Gaussian Error Linear Unit)

DIAGRAM

```
GELU(x) ≈ 0.5 * x * (1 + tanh(√(2/π) * (x + 0.044715 * x³)))
```

- Smooth, non-monotonic around zero
- Better gradient properties than ReLU in practice
- **Use:** Transformers (BERT, GPT), modern architectures
- **Intuition:** Stochastically gates input based on its magnitude

Softmax

DIAGRAM

```
softmax(x_i) = e^(x_i) / sum_j(e^(x_j))
```

- Outputs sum to 1 → probability distribution over classes
- Used only in **output layer** for multi-class classification
- Pairs with cross-entropy loss
- **Numerical stability trick:** subtract max(x) before exponentiating

Activation Function Comparison Table

Activation	Range	Zero-Centered	Vanishing Grad	Dead Neurons	Primary Use
Sigmoid	(0,1)	No	Yes (severe)	No	Binary output
Tanh	(-1,1)	Yes	Yes (mild)	No	RNN hidden states
ReLU	[0,∞)	No	No (pos side)	Yes	Hidden layers (default)
Leaky ReLU	(-∞,∞)	No	No	No	When dead neurons occur
GELU	(-∞,∞)	No	No	No	Transformers, modern nets
Softmax	(0,1), sum=1	No	N/A	N/A	Multiclass output

Why non-linearity matters — the collapse argument:

```python
# WITHOUT activation (just linear layers):
h1 = W1 @ x + b1
h2 = W2 @ h1 + b2
# h2 = W2 @ (W1 @ x + b1) + b2
#    = (W2@W1) @ x + (W2@b1 + b2)
#    = W_eff @ x + b_eff   ← still linear!

# WITH activation:
h1 = relu(W1 @ x + b1)
h2 = relu(W2 @ h1 + b2)
# Cannot be collapsed → genuinely non-linear function
```

Forward Propagation

The **forward pass** computes the network's output from input to prediction:

```python
def forward(x, layers):
    """
    x: input tensor [batch_size, input_dim]
    layers: list of (W, b, activation) tuples
    """
    h = x
    for W, b, act in layers:
        z = h @ W + b        # linear transformation
        h = act(z)           # non-linear activation
    return h                 # final output (logits or probabilities)
```

Key tensors: - `z^(l) = W^(l) * a^(l-1) + b^(l)` — pre-activation (logit) - `a^(l) = f(z^(l))` — post-activation (activation)

Store both z and a for each layer — needed in backprop.

Loss Functions

The loss function measures how wrong the network's predictions are. It must be **differentiable** so gradients can flow.

Mean Squared Error (MSE) — Regression

DIAGRAM

```
MSE = (1/n) * sum_i( (y_i - y_hat_i)^2 )
```

- Penalizes large errors quadratically
- Sensitive to outliers (use MAE if outliers are common)
- Gradient: `dL/dy_hat = -2/n * (y - y_hat)`

Binary Cross-Entropy — Binary Classification

DIAGRAM

```
BCE = -( y * log(y_hat) + (1-y) * log(1-y_hat) )
```

- Assumes `y_hat = sigmoid(logit)` ∈ (0,1)
- Penalizes confident wrong predictions heavily (log of near-zero → -∞)

Categorical Cross-Entropy — Multiclass

DIAGRAM

```
CE = -sum_c( y_c * log(y_hat_c) )
```

- y is one-hot encoded, `y_hat = softmax(logits)`
- In practice: only the log of the predicted probability for the true class matters
- Combined with softmax → "softmax + cross-entropy" (numerically stable together)

Cross-Entropy vs MSE for Classification

	Cross-Entropy	MSE
Appropriate for	Classification	Regression
Gradient behavior	Strong gradient even when very wrong	Weak gradient when output is saturated
Output activation	Softmax/Sigmoid	Linear
Why CE is better for classification	Log loss aligns with MLE for categorical distributions	Gradients vanish with sigmoid/softmax output when error is large

Backpropagation & the Chain Rule

Backpropagation is **automatic differentiation applied to a computational graph** — computing `dL/dW` for every weight `W` using the chain rule of calculus.

The chain rule:

DIAGRAM

```
If y = f(g(x)), then dy/dx = dy/dg * dg/dx

More generally for composed functions:
dL/dW^(l) = dL/da^(l) * da^(l)/dz^(l) * dz^(l)/dW^(l)
```

Backprop algorithm:

PYTHON

```python
def backward(loss, layers, stored_activations):
    """
    Compute gradients for all weights using chain rule.
    stored_activations: (z^l, a^l) for each layer from forward pass
    """
    dL_da = d_loss_d_output(loss)           # gradient of loss w.r.t. output

    for l in reversed(range(len(layers))):
        z_l, a_prev = stored_activations[l]
        W_l, b_l, act_l = layers[l]

        # Step 1: gradient through activation function
        dL_dz = dL_da * act_l.derivative(z_l)   # element-wise

        # Step 2: gradient w.r.t. weights and bias
        dL_dW = a_prev.T @ dL_dz                # [n_in, batch] @ [batch, n_out]
        dL_db = dL_dz.sum(axis=0)               # sum over batch

        # Step 3: gradient to pass to previous layer
        dL_da = dL_dz @ W_l.T                   # [batch, n_in]

        # Store for optimizer
        gradients[l] = (dL_dW, dL_db)

    return gradients
```

Key insight: The gradient "flows backward" through each layer. Each layer contributes a factor to the chain — if any factor is near zero (saturated activation), the gradient vanishes.

Interview takeaway: Backprop is just the chain rule applied layer by layer. The "backward" graph is the transpose of the forward graph.

Optimizers

Optimizers determine **how** gradient information is used to update weights.

SGD (Stochastic Gradient Descent)

DIAGRAM

```
W = W - lr * dL/dW
```

- Simple, well-understood, good generalization
- **Problem:** Single fixed learning rate, slow convergence, oscillates in narrow ravines
- **Stochastic** = uses mini-batches (random subset), not full dataset — noisy but faster

SGD with Momentum

DIAGRAM

```
v_t = β * v_{t-1} + (1-β) * dL/dW_t
W = W - lr * v_t
```

- v_t = velocity (exponential moving average of gradients)
- β typically 0.9
- **Effect:** Accelerates in consistent gradient directions, dampens oscillations
- **Analogy:** A ball rolling downhill — it builds momentum in the right direction

RMSprop

DIAGRAM

```
s_t = β * s_{t-1} + (1-β) * (dL/dW_t)²
W = W - (lr / sqrt(s_t + ε)) * dL/dW_t
```

- Adapts learning rate **per parameter** based on recent gradient magnitudes
- Parameters with large gradients get smaller effective LR
- Good for RNNs and non-stationary objectives

Adam (Adaptive Moment Estimation)

ADAM OPTIMIZER · ADAPTIVE MOMENT ESTIMATION

1st Moment $\hat{m}$
bias-corrected mean of gradients · momentum

+

2nd Moment $\hat{v}$
bias-corrected mean of grad² · adaptive scale

↓ combined update ↓

$$W = W - lr \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$$

each parameter gets its own effective learning rate

◆ KEY EQUATIONS

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t$$

$$v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$$

$$\hat{m} = m_t / (1-\beta_1^t) \quad \hat{v} = v_t / (1-\beta_2^t)$$

$$W = W - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon)$$

Default: β_1=0.9, β_2=0.999, ε=1e-8. Bias-correction prevents large steps at initialisation.

- **β1=0.9, β2=0.999, ε=1e-8** (defaults)
- Combines momentum (1st moment) + per-parameter LR adaptation (2nd moment)
- Bias correction handles initialization bias (m_0 = v_0 = 0)
- **Default choice for most tasks**
- **Weakness:** Can generalize slightly worse than SGD+momentum on some tasks (Adam converges faster but to a slightly different minimum)

Learning Rate Schedules

Schedule	Formula / Description	Use Case
Step Decay	LR *= γ every N epochs	Simple, works well
Cosine Annealing	LR = LR_min + 0.5*(LR_max-LR_min)*(1+cos(π*t/T))	Transformers, fine-tuning
Warmup + Decay	Linear warmup for K steps, then decay	Large model training (GPT, BERT)
Cyclic LR	Oscillate between LR_min and LR_max	Escape local minima
ReduceLROnPlateau	Reduce LR when metric stops improving	When you don't know when to decay

Why warmup? Large models are unstable early in training — gradients are noisy, momentum is uninitialized. Starting with a small LR and ramping up prevents early divergence.

Optimizer Comparison

Optimizer	Adapts LR?	Momentum?	Memory	Best For
SGD	No	No	1x params	Final fine-tuning, convex problems
SGD+Momentum	No	Yes	2x params	Vision models (ResNet, etc.)
RMSprop	Yes	No	2x params	RNNs, non-stationary
Adam	Yes	Yes	3x params	Default: most deep learning tasks
AdamW	Yes	Yes	3x params	Transformers (decouples weight decay)

Vanishing & Exploding Gradients

Vanishing gradients: During backprop, gradients become exponentially small as they propagate through many layers → early layers learn very slowly or not at all.

Root cause: Chain rule multiplies many numbers together. If each factor < 1 (e.g., sigmoid derivative max = 0.25), the product shrinks exponentially with depth.

DIAGRAM

```
dL/dW^(1) = dL/da^(L) * (da^(L)/dz^(L)) * ... * (da^(2)/dz^(2)) * (da^(1)/dz^(1))
          = product of L terms, each ≤ 0.25 for sigmoid
          ≈ 0.25^L → 0 as L → ∞
```

Exploding gradients: Same mechanism but factors > 1 → gradients blow up → NaN weights.

Solutions:

Problem	Solution	Mechanism
Vanishing	ReLU activation	Derivative = 1 for x > 0, doesn't shrink
Vanishing	Residual connections (ResNets)	Gradient highway: `dL/dW^(l)` gets additive term of 1
Vanishing	Better initialization	Xavier/He — keep gradient magnitude stable
Vanishing	Batch normalization	Normalizes activations, keeps them in active range
Exploding	Gradient clipping	`g = g * (max_norm / ||g||)` if `||g|| > max_norm`
Both	LSTM/GRU gating	Additive cell state update instead of multiplicative

Interview takeaway: ReLU + residual connections + BatchNorm together largely solved vanishing gradients for feedforward nets. LSTMs solved it for RNNs.

Weight Initialization

Poor initialization → dead neurons, vanishing/exploding gradients from step 1 (before training even starts).

Goal: Keep the variance of activations and gradients approximately constant across layers.

Xavier/Glorot Initialization (for Tanh/Sigmoid)

DIAGRAM

```
W ~ Uniform(-√(6/(n_in + n_out)), √(6/(n_in + n_out)))
# or equivalently:
W ~ Normal(0, √(2/(n_in + n_out)))
```

- Derived by requiring `Var(output) = Var(input)` for linear activations
- Works well with tanh and sigmoid

He Initialization (for ReLU)

DIAGRAM

```
W ~ Normal(0, √(2/n_in))
```

- Accounts for ReLU zeroing out half the inputs (halves the effective variance)
- **Use He init with ReLU networks**

Why initialization matters

DIAGRAM

```
# Bad init: all zeros
W = 0 → all neurons compute same output → same gradient → stay identical forever
# "Symmetry breaking" is essential — init must be random

# Bad init: too large
activations → saturate → vanishing gradients from step 1

# Bad init: too small
activations → near zero → gradients near zero → barely learns
```

Interview takeaway: Xavier for tanh/sigmoid, He for ReLU. Always random (never all-zeros for weights — zeros for biases is fine).

Batch Normalization & Layer Normalization

Batch Normalization (BatchNorm)

Normalize activations **across the batch dimension** for each feature:

DIAGRAM

```
μ_B = (1/m) * sum_i(x_i)          # batch mean
σ²_B = (1/m) * sum_i((x_i - μ_B)²) # batch variance

x_hat_i = (x_i - μ_B) / sqrt(σ²_B + ε)   # normalize

y_i = γ * x_hat_i + β             # learnable scale and shift
```

- γ, β are learned parameters (lets network un-normalize if needed)
- Applied **before** the activation function (or after — debate exists)
- At inference: use running mean/variance tracked during training (not batch stats)

Benefits: 1. Reduces internal covariate shift 2. Allows higher learning rates 3. Acts as slight regularizer (noise from batch statistics) 4. Reduces sensitivity to initialization

Problems with BatchNorm: - Batch size dependent — fails for batch_size=1 - Can't be used as-is for RNNs (variable-length sequences) - Creates dependency between samples in a batch

Layer Normalization (LayerNorm)

Normalize **across the feature dimension** for each sample:

DIAGRAM

```
μ = (1/H) * sum_j(x_j)              # mean over features
σ² = (1/H) * sum_j((x_j - μ)²)    # variance over features

x_hat = (x - μ) / sqrt(σ² + ε)
y = γ * x_hat + β
```

- Independent of batch size
- Same behavior at train and inference time
- **Works for RNNs and Transformers**
- Default in all Transformer architectures (BERT, GPT, etc.)

BatchNorm vs LayerNorm

	BatchNorm	LayerNorm
Normalization axis	Across batch, per feature	Across features, per sample
Batch size dependent	Yes	No
Works for RNNs/Transformers	No	Yes
Used in	CNNs, MLPs	RNNs, Transformers
Train/inference difference	Yes (running stats)	No
Acts as regularizer	Yes (batch noise)	Minimal

Dropout & Regularization in Deep Learning

Dropout

During training, randomly set each neuron's output to **zero with probability p** (typically p=0.2 to 0.5):

PYTHON

```python
def dropout(x, p, training=True):
    if not training:
        return x                          # No dropout at inference
    mask = (torch.rand_like(x) > p)       # Bernoulli mask
    return x * mask / (1 - p)             # Scale to maintain expected value
```

Why it works: 1. Prevents co-adaptation — neurons can't rely on specific other neurons 2. Equivalent to training an ensemble of 2^n different network architectures 3. At inference, full network with scaled weights approximates ensemble average

Where to apply: After fully-connected layers, NOT typically after convolutions (or use spatial dropout for CNNs).

Other DL Regularization Techniques

Technique	Mechanism	When to Use
L2 (Weight Decay)	`loss += λ * sum(W²)` → pulls weights toward zero	Always, as AdamW
L1	`loss += λ * sum(|W|)` → induces sparsity	Sparse networks
Dropout	Random neuron deactivation	FC layers, Transformers
Data Augmentation	Random crops, flips, color jitter	Image tasks
Early Stopping	Stop when val loss stops improving	All tasks
Label Smoothing	`y = (1-ε)*y_onehot + ε/K`	Classification, prevents overconfidence
Batch Norm	Noise from batch statistics	CNNs

Convolutional Neural Networks (CNNs)

CNNs exploit **spatial structure** — nearby pixels are correlated. Instead of connecting every input pixel to every neuron (which would be computationally infeasible for images), use **local, shared filters**.

Core Components

Convolution Operation:

DIAGRAM

```
(f * I)[i,j] = sum_m sum_n f[m,n] * I[i+m, j+n]
```

- **Filter/Kernel:** Small weight matrix (e.g., 3×3, 5×5) that slides over input
- **Feature map:** Output of applying one filter to the full input
- **Multiple filters** → multiple feature maps → detect different patterns
- **Shared weights:** Same filter weights applied at every position → translation invariance + far fewer parameters than FC

Stride: How many pixels the filter moves each step. - Stride 1: output ≈ same size as input - Stride 2: halves spatial dimensions

Padding: - "Valid": no padding → output smaller than input - "Same": pad with zeros → output same size as input

Output size formula:

DIAGRAM

```
output_size = floor((input_size - kernel_size + 2*padding) / stride) + 1
```

Pooling: - Max pooling: takes maximum in each local region → spatial downsampling + translation invariance - Average pooling: takes average - Global Average Pooling (GAP): pools entire feature map to single value per channel

Receptive Field: The area of the original input that influences one output neuron. Deeper layers have larger receptive fields.

Classic CNN Architectures

Architecture	Year	Key Innovation	Params
LeNet-5	1998	Conv-Pool-FC design	~60K
AlexNet	2012	Deep CNN + ReLU + Dropout + GPU	~60M
VGG-16	2014	Uniform 3×3 convs, very deep	~138M
GoogLeNet/Inception	2014	Inception modules (multi-scale filters)	~6.8M
ResNet-50	2015	Residual connections → very deep (50-152 layers)	~25M
EfficientNet	2019	Compound scaling (depth, width, resolution)	5-66M

ResNet key innovation — skip connections:

DIAGRAM

```
h = F(x, W) + x    # Instead of just h = F(x, W)
```

The `+ x` creates a "gradient highway" — backprop can flow directly through the identity connection, bypassing potentially vanishing layers.

Parameter Count for Convolution Layer

DIAGRAM

```
params = (kernel_h * kernel_w * in_channels + 1) * out_channels
#         weights per filter                      number of filters
#         +1 for bias
```

Compare to FC layer: `params = in_size * out_size + out_size`. For images, conv is massively more parameter-efficient.

Recurrent Neural Networks (RNNs)

RNNs process **sequential data** by maintaining a hidden state that carries information across time steps:

```python
def rnn_step(x_t, h_prev, W_xh, W_hh, b_h, W_hy, b_y):
    h_t = tanh(W_xh @ x_t + W_hh @ h_prev + b_h)  # hidden state update
    y_t = W_hy @ h_t + b_y                        # output (optional per step)
    return h_t, y_t
```

The same weights (W_xh, W_hh) are shared across all time steps — this is analogous to weight sharing in CNNs.

Problems with vanilla RNNs: 1. **Vanishing gradients through time:** dL/dh_0 = product of Jacobians across T steps → exponentially small for long sequences 2. **Exploding gradients:** Same mechanism in reverse 3. **Short-term memory:** Effectively forgets information from >10-20 steps back

LSTM (Long Short-Term Memory)

LSTMs solve the vanishing gradient problem with a **cell state** (long-term memory) and **gating mechanisms**:

```python
# Gates (all computed same way: sigmoid of linear combination)
i_t = sigmoid(W_i @ [h_{t-1}, x_t] + b_i)   # input gate: what to write
f_t = sigmoid(W_f @ [h_{t-1}, x_t] + b_f)   # forget gate: what to erase
o_t = sigmoid(W_o @ [h_{t-1}, x_t] + b_o)   # output gate: what to read

# Candidate cell state
c_hat_t = tanh(W_c @ [h_{t-1}, x_t] + b_c)

# Cell state update (ADDITIVE — preserves gradient flow!)
c_t = f_t * c_{t-1} + i_t * c_hat_t

# Hidden state
h_t = o_t * tanh(c_t)
```

Why LSTMs work: - **Additive cell update:** `c_t = f_t * c_{t-1} + i_t * c_hat_t` — gradient can flow through addition without multiplication, analogous to ResNet skip connections - **Forget gate ≈ 1:** If forget gate is near 1, cell state is preserved unchanged, gradient flows unimpeded - Separate `h_t` (working memory) and `c_t` (long-term memory)

GRU (Gated Recurrent Unit)

Simplified LSTM with two gates instead of three:

```python
z_t = sigmoid(W_z @ [h_{t-1}, x_t])   # update gate
r_t = sigmoid(W_r @ [h_{t-1}, x_t])   # reset gate

h_hat_t = tanh(W @ [r_t * h_{t-1}, x_t])  # candidate

h_t = (1 - z_t) * h_{t-1} + z_t * h_hat_t  # final hidden state
```

- Merges cell state and hidden state into single `h_t`
- Fewer parameters → faster to train
- Comparable performance to LSTM in practice

RNN vs LSTM vs GRU

	Vanilla RNN	LSTM	GRU
Parameters	Fewest	Most	Middle
Memory length	Short	Long	Long
Training stability	Poor	Good	Good
Speed	Fastest	Slowest	Middle
Gates	0	3 (i,f,o)	2 (z,r)
When to use	Rarely	Long sequences	Long sequences, faster training

Embeddings & Word2Vec

Embedding: A dense, low-dimensional vector representation of a discrete entity (word, user, product).

Why not one-hot? For vocabulary of 100K words, one-hot is 100K-dimensional, sparse, and captures no semantic similarity.

Word2Vec intuition: - Train a neural network to predict: given a word, predict its context (Skip-gram) or given context, predict center word (CBOW) - The intermediate representation (the weight matrix to the hidden layer) = word embeddings - After training: semantically similar words have similar vectors - Famous example: `king - man + woman ≈ queen` (vector arithmetic captures analogy)

Two architectures: - **CBOW (Continuous Bag of Words):** Context → center word. Faster, good for frequent words. - **Skip-gram:** Center → context words. Slower but better for rare words.

Modern embeddings: Word2Vec is conceptual predecessor to contextual embeddings (ELMo, BERT) — same word gets different embedding depending on context (captures polysemy).

Interview takeaway: An embedding layer is just a lookup table (a matrix of size `vocab_size × embed_dim`) — `nn.Embedding(vocab_size, d)` in PyTorch is literally a matrix, indexed by integer word IDs.

Transfer Learning & Fine-Tuning

Transfer learning: Use a model pre-trained on a large dataset (ImageNet, web text) as starting point for a different (usually smaller) task.

Why it works: Early layers learn universal features (edges, curves, basic patterns) that transfer across tasks. Later layers learn task-specific features that can be replaced or fine-tuned.

Strategies

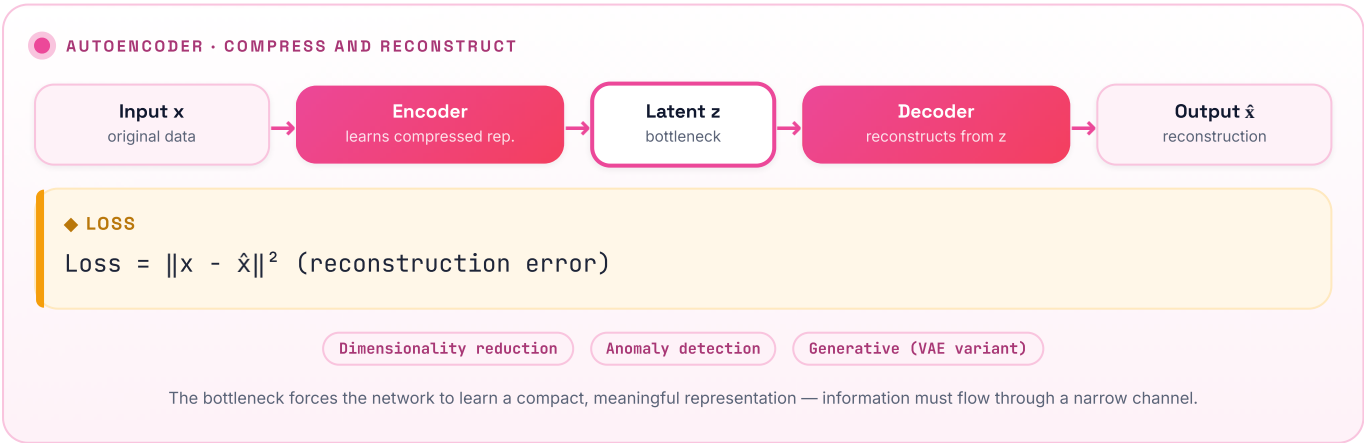
Strategy	What's Frozen	What's Trained	When to Use
Feature extraction	All pre-trained layers	Only new head	Small dataset, very similar domain
Fine-tuning (shallow)	Early layers	Later layers + head	Medium dataset, somewhat similar domain
Full fine-tuning	Nothing	Entire network	Large dataset or very different domain
LoRA/PEFT	Most weights	Low-rank adapters	LLM fine-tuning, minimal compute

Key concepts: - **Pre-trained model:** Trained on large, general dataset (ImageNet for vision, BookCorpus/Wikipedia for NLP) - **Downstream task:** Your specific task (medical image classification, sentiment analysis) - **Head:** The final layer(s) replaced for the new task (e.g., new linear classifier) - **Learning rate:** Use much smaller LR for pre-trained layers (1e-5 to 1e-4) vs new layers (1e-3)

Interview takeaway: Transfer learning is one of the most practically important concepts in modern ML — almost no one trains from scratch at FAANG; everything starts from a pre-trained checkpoint.

Autoencoders

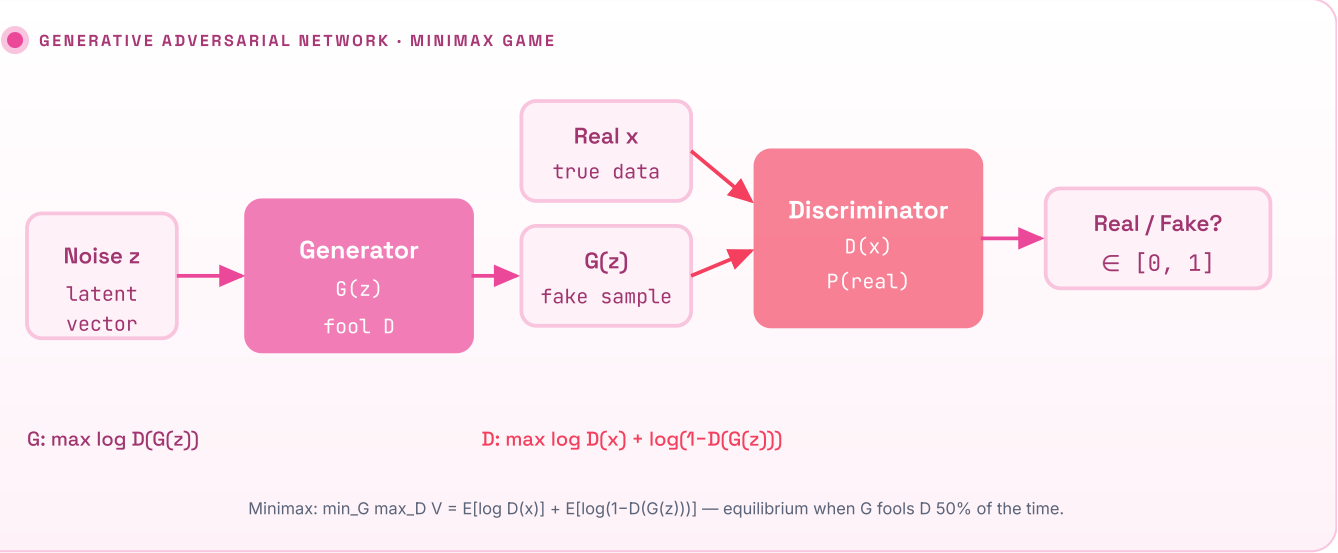
An **autoencoder** is a neural network trained to compress its input into a low-dimensional **latent representation** (encoding) and then reconstruct the original input (decoding):



Why it's useful: - Learns unsupervised feature representations - Anomaly detection: reconstruction error high for anomalous inputs - Dimensionality reduction (non-linear PCA) - **Variational Autoencoder (VAE):** Latent space is a probability distribution (Normal) → enables generation. Loss adds KL-divergence term.

Generative Adversarial Networks (GANs)

Two networks competing in a minimax game:



Training loop: 1. Sample real data batch x 2. Sample noise z, generate fake data `G(z)` 3. Train D to distinguish real from fake 4. Train G to fool D (freeze D's weights) 5. Repeat

Key challenges: - **Mode collapse:** G learns to generate only a few modes (types) of data - **Training instability:** G and D need to progress together - **Evaluation:** FID (Fréchet Inception Distance) score

Modern successors: Diffusion models have largely replaced GANs for image generation (DALL-E, Stable Diffusion) due to training stability.

Overfitting in Deep Learning

Overfitting: Model memorizes training data, performs poorly on unseen data. Neural nets are especially prone — they have massive capacity.

Diagnosing:

DIAGRAM

```
Training loss: 0.01
Validation loss: 2.5  → Classic overfit
```

Solutions in DL:

Technique	Effect
More data	Best solution; data augmentation as proxy
Dropout	Ensemble effect, breaks co-adaptation
Weight decay (L2)	Smaller weights → smoother function
Early stopping	Stop before model memorizes training set
Batch normalization	Mild regularization via batch noise
Reduce model capacity	Fewer layers/neurons
Data augmentation	Random transformations → effective more data
Label smoothing	Prevents overconfident predictions

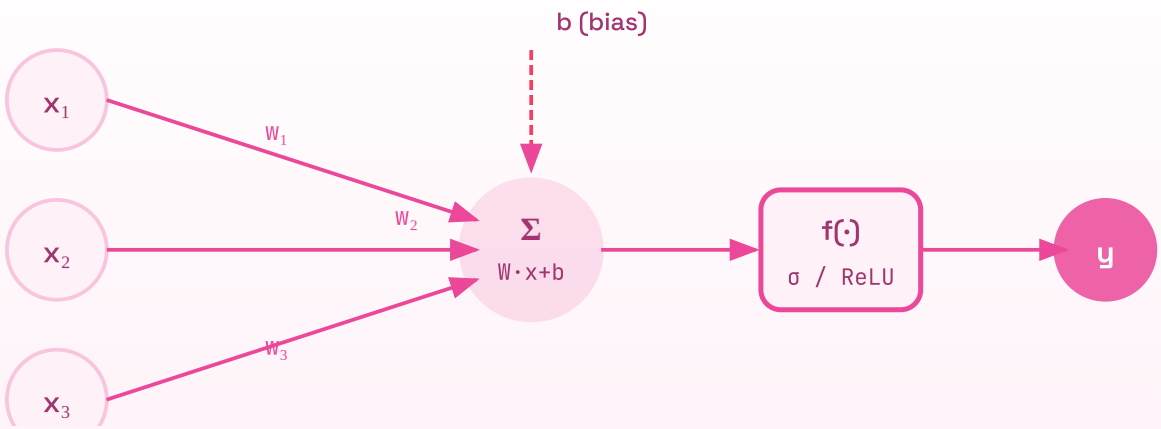
Bias-variance in DL: - Underfitting = high bias → need bigger model, more training - Overfitting = high variance → need regularization, more data - Deep learning has largely killed the classical bias-variance trade-off concern at scale — modern large models can simultaneously have low bias AND low variance if trained with enough data (the "double descent" phenomenon)

SECTION 05

05 Architecture / Diagrams

Single Neuron / Perceptron

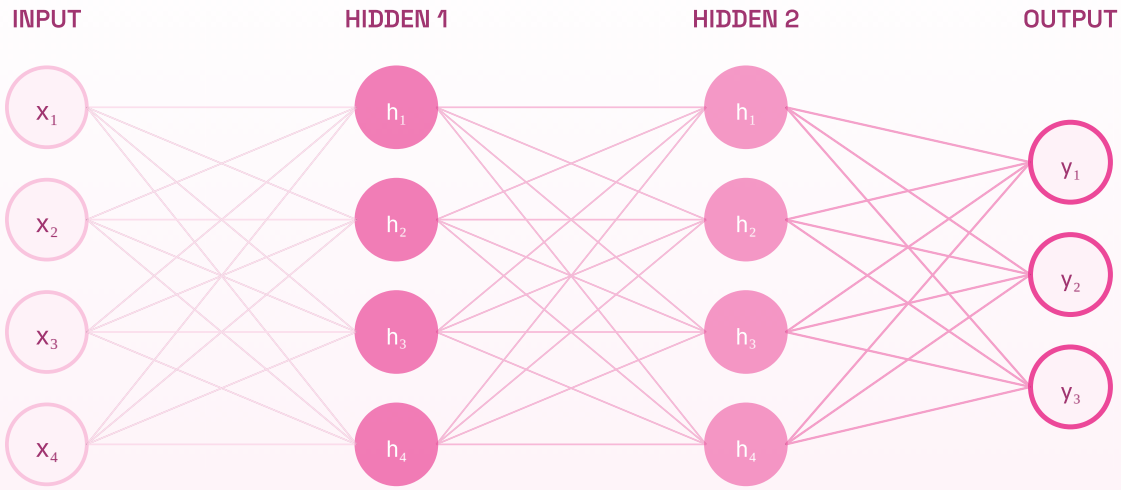
SINGLE NEURON · LINEAR COMBINATION + ACTIVATION



y = f(w₁x₁ + w₂x₂ + w₃x₃ + b) — the fundamental building block of all neural networks.

MLP Architecture

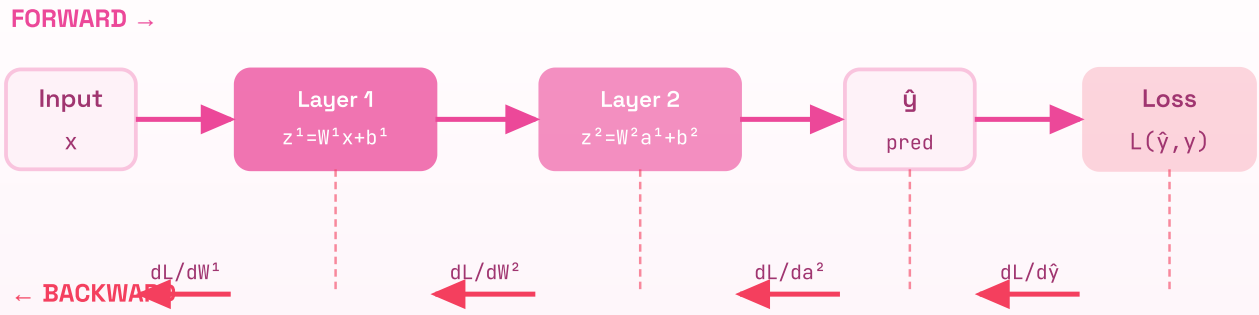
MULTI-LAYER PERCEPTRON · FULLY CONNECTED LAYERS



Every node in one layer connects to every node in the next (fully-connected). Each edge is a learnable weight.

Forward + Backpropagation Flow

FORWARD PASS & BACKPROPAGATION · CHAIN RULE

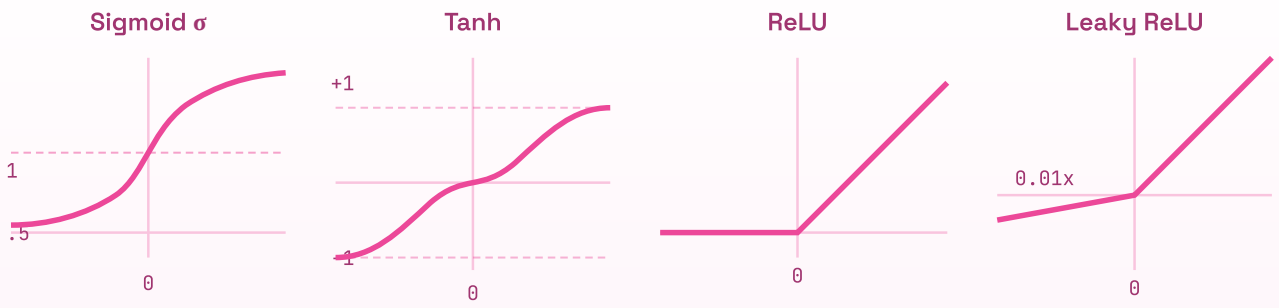


$$dL/dW^2 = dL/da^2 \cdot f'(z^2) \cdot a^{1T}$$

Forward pass computes predictions left→right; backward pass propagates gradients right→left via the chain rule.

Activation Function Shapes

ACTIVATION FUNCTIONS · SHAPE COMPARISON

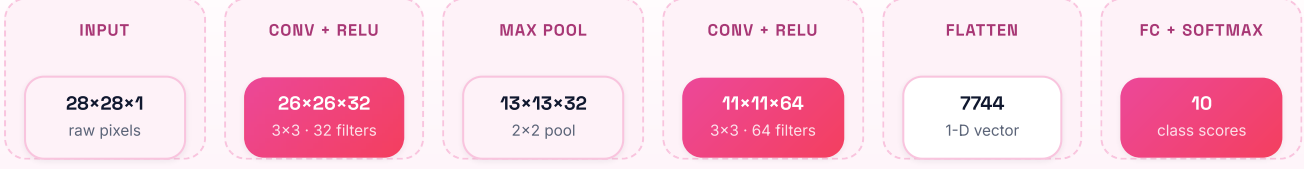


`Sigmoid: (0,1) — saturates` `Tanh: (-1,1) — zero-centered` `ReLU: max(0,x) — dead for x<0` `LeakyReLU: small grad for x<0`

ReLU and its variants dominate modern networks; sigmoid/tanh cause vanishing gradients in deep stacks.

CNN Architecture

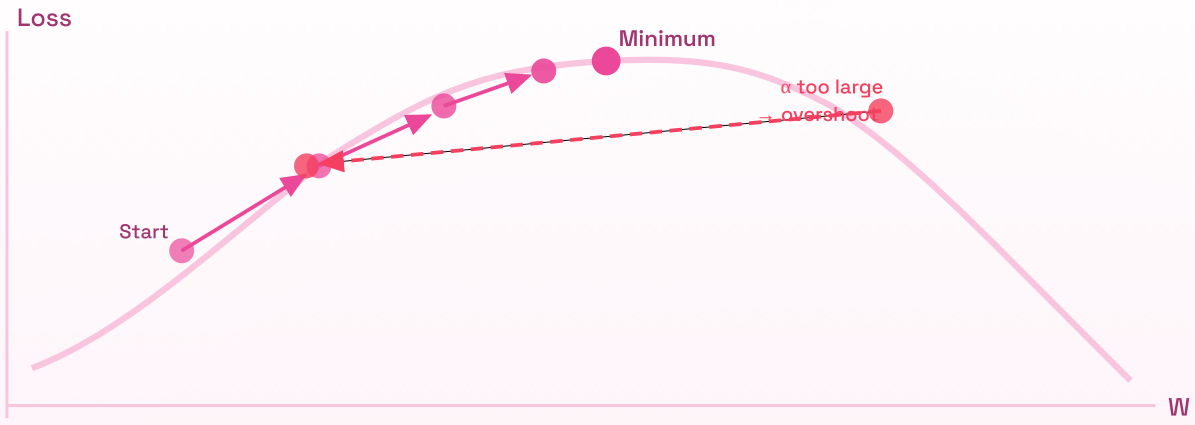
CONVOLUTIONAL NEURAL NETWORK · IMAGE CLASSIFICATION PIPELINE



Each conv layer slides learnable filters across the spatial input — earlier layers detect edges, later layers detect semantics.

Gradient Descent Visualization

GRADIENT DESCENT · NAVIGATING THE LOSS LANDSCAPE



$$\theta_{t+1} = \theta_t - \alpha \cdot \nabla L(\theta_t)$$

α too large → diverge / oscillate

α too small → very slow

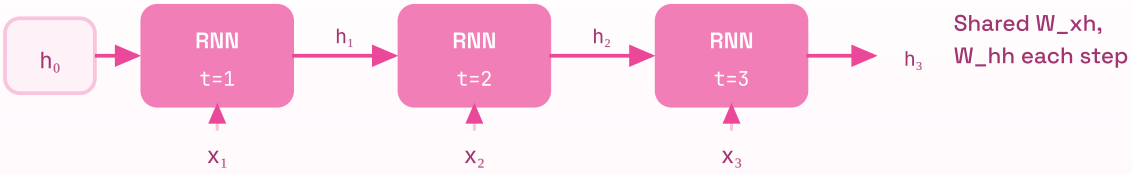
Adam → adaptive α per param

Each step moves weights opposite to the gradient direction, scaled by learning rate α.

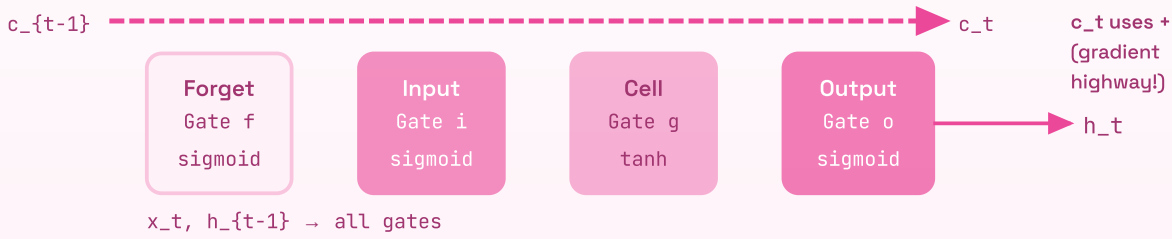
RNN / LSTM Unrolled

RECURRENT NETWORK · UNROLLED RNN & LSTM CELL

Vanilla RNN (unrolled across time)



LSTM Cell · t



LSTM replaces vanishing-gradient multiplication with additive cell-state updates — the key to long-range memory.

SECTION 06

06 Real-World Examples

Company	Application	Architecture	Key Technique
Google Photos	Image search/classification	EfficientNet/ViT	CNN pre-trained on JFT-300M
Netflix	Thumbnail CTR optimization	CNN	Transfer from ImageNet
Amazon Alexa	Wake word detection	Small CNN/RNN	Trained to run on device
YouTube	Video recommendation	Two-Tower DNN	Embeddings for users and videos
Facebook	Hate speech detection	BERT fine-tuned	Transfer learning
Tesla	Autopilot vision	Multi-camera CNN	Custom hardware (FSD chip)
Spotify	Song recommendations	Autoencoder + collab filtering	Learned audio embeddings
OpenAI DALL-E	Image generation	Diffusion + CLIP	Contrastive + generative

SECTION 07

07 Real-Life Analogies — The Learning Factory Assembly Line

Every component of deep learning maps onto a factory that manufactures products and learns by inspecting its own output quality.

DL Concept	Factory Analogy
Input data	Raw materials entering the factory
Neural network	The entire factory assembly line
Layer	One station on the assembly line (e.g., cutting, welding, painting)
Weights	The dial settings at each station (speed, temperature, pressure)
Forward pass	The product traveling down the line, being transformed at each station
Activation function	A station's go/no-go quality gate — product passes or gets modified
Loss function	The final Quality Control (QC) inspector's defect score
Backpropagation	The foreman walking the line **backward** from QC, telling each station specifically how its settings caused the defect
Gradient	How much each station's dials need to change to reduce defects
Gradient descent	The foreman nudging each dial a small amount in the right direction
Learning rate	How big a nudge the foreman applies (too big = overcompensate; too small = too slow)
Adam optimizer	A smart foreman who tracks which dials have been moved a lot recently and adjusts nudge size per-dial accordingly
Vanishing gradient	By the time the foreman reaches station 1 (near the start), his feedback has been diluted so many times he can only whisper — station 1 barely learns
Residual connections	A direct phone line from QC to every station, so even station 1 gets clear feedback
Dropout	Randomly sending workers home each day so the line can't over-rely on any one person — forces robustness
Batch normalization	Standardizing the product dimensions at each station so later stations don't have to compensate for upstream variation
CNN	Inspectors using a small magnifying glass that slides across the product surface, looking for defects in each patch — same inspector logic applied everywhere
RNN	A line that remembers the last product it processed, using that memory to better process the current one
LSTM	A line with a long-term clipboard (cell state) that records important product history, with explicit "write/erase/read" decisions
Transfer learning	Hiring an experienced foreman from another factory (pre-trained on a related product) — retrain only the final stations for your specific product
Overfitting	The line gets so optimized for the specific training products it's seen that it fails on new, slightly different products
Embeddings	Each product type gets a detailed spec sheet (dense vector) summarizing its properties for downstream stations

SECTION 08

08 Memory Tricks / Mnemonics

Activation functions — "STRL GS": - **S**igmoid: 0-1 output, squashes, **S**aturates - **T**anh: -1 to 1, **T**wo-sided sigmoid, zero-centered - **R**eLU: **R**ectify negatives to zero, **R**isk of dead neurons - **L**eaky ReLU: **L**ittle slope for negatives - **G**ELU: **G**aussian-flavored, used in **G**PT - **S**oftmax: **S**um to 1, for **S**election (multiclass)

Optimizer progression — "S M R A": - **S**GD → add **M**omentum → add per-param LR (**R**MSprop) → combine both (**A**dam)

LSTM gates — "I F O" (IFO = "I Forget Output"): - **I**nput gate: what new info to write to cell - **F**orget gate: what old info to erase from cell - **O**utput gate: what to read from cell as hidden state

Weight init — "Xavier for tanh, He for ReLU": - Xavier → "X" looks like a cross → tanh crosses zero - He → "H" → Half inputs are zero (ReLU), so scale by 2/n

BatchNorm vs LayerNorm — "Batch = across samples, Layer = across features": - **B**atch: **B**ig dimension (across many samples per feature) - **L**ayer: **L**ength of the feature vector (across all features per sample)

CNN parameter count formula — "K K C + 1 times F": - (Kernel_h × Kernel_w × Channels_in + 1) × Filters_out

Backprop chain rule — "LOCAL × UPSTREAM": - At each node: gradient = local derivative × gradient from downstream - `dL/dW = (dL/da_out) × (da_out/dz) × (dz/dW)` - = upstream gradient × activation derivative × input

Vanishing gradient solutions — "RICH": - **R**eLU (better activation) - **I**nitialization (Xavier/He) - **C**onnections (residual/skip) - **H**normalization (batch/layer norm)

SECTION 09

09 Common Interview Questions

Q1: Explain backpropagation from first principles.

Model answer: Backpropagation computes gradients of the loss with respect to all parameters using the chain rule of calculus. After the forward pass computes the loss, we propagate the error signal backward through the computational graph.

At each layer, we compute three things: 1. `dL/dz^l = dL/da^l * f'(z^l)` — gradient through activation 2. `dL/dW^l = a^(l-1)^T * dL/dz^l` — gradient for weight update 3. `dL/da^(l-1) = dL/dz^l * W^l^T` — gradient to pass to previous layer

The key insight: each layer's gradient depends only on the next layer's gradient (upstream) and its own local derivatives. We store forward pass activations and reuse them in the backward pass.

Follow-up: "Why do we need to store activations from the forward pass?" - We need `a^(l-1)` to compute `dL/dW^l = a^(l-1)^T * dL/dz^l` - We need `z^l` to compute `f'(z^l)` (the activation derivative) - Memory cost of training = O(num_layers × batch_size × layer_size) for stored activations - Gradient checkpointing trades compute for memory by recomputing activations

Q2: Why does ReLU work better than sigmoid for hidden layers?

Model answer: Three reasons:

1. **No vanishing gradient in the positive region.** Sigmoid derivative max is 0.25 — multiply that through 50 layers and the gradient ≈ 0. ReLU derivative for x>0 is exactly 1, so gradients flow without shrinking.
2. **Computationally cheap.** ReLU is just `max(0, x)` — a single comparison. Sigmoid requires an exponentiation.
3. **Sparse activation.** Only ~50% of neurons activate for any given input. Sparse representations are more computationally efficient and have been linked to better generalization.

Trade-off: Dead neurons — if a neuron's weights cause it to always receive negative inputs, gradient = 0 forever, weights never update. Solutions: Leaky ReLU, careful initialization, lower learning rate.

Follow-up: "When would you still use sigmoid?" - Output layer for binary classification (need probability in [0,1]) - LSTM gates (need to produce values in [0,1] for gating)

Q3: Explain the vanishing gradient problem and how ResNets solve it.

Model answer: In backprop, the gradient at layer `l` is the product of Jacobians from layer `l` to the output. With L layers of sigmoid (max derivative 0.25), the gradient magnitude scales as 0.25^L. For L=20, that's ~10^-12 — effectively zero. Early layers learn nothing.

ResNets add skip connections: `output = F(x, W) + x`

During backprop: `dL/dx = dL/d(output) * (dF/dx + I)`

The `I` (identity) term means there's always a direct gradient path through the skip connection — gradient = upstream + local. Even if `dF/dx` is small, the upstream gradient flows unimpeded. This lets ResNets be 50-152 layers deep without gradient issues.

Analogy: Like adding a fiber optic cable directly from the foreman's office to every station, bypassing the chain of verbal relays.

Q4: Compare Adam vs SGD. When would you use each?

Model answer:

Adam advantages: - Adaptive per-parameter learning rates — features with rare but informative gradients (like word embeddings for rare words) get appropriate updates - Incorporates momentum automatically - Requires less tuning — default hyperparameters work well in most cases - Converges faster in wall-clock time

SGD+Momentum advantages: - Often generalizes slightly better (converges to flatter minima) - Lower memory usage (2x params vs 3x for Adam) - Better understood theoretically

When to use each: - Adam: Default choice, especially for: NLP tasks, transformers, when prototyping, when data is sparse - SGD+Momentum: When ultimate accuracy matters and you can afford more tuning (ResNet image classifiers), fine-tuning of pre-trained models

AdamW: Adam with proper weight decay (decoupled from gradient update). Use this for transformer fine-tuning — standard Adam's weight decay is mathematically incorrect (it interacts with the adaptive LR).

Q5: Why does dropout work as regularization?

Model answer: Two complementary explanations:

1. **Ensemble interpretation:** With p=0.5 dropout on n neurons, there are 2^n possible sub-networks. Training with dropout trains all of them simultaneously with shared weights. At inference, the full network (with weights scaled by 1-p) approximates the average of this ensemble.

2. **Co-adaptation prevention:** Without dropout, neurons can develop complex codependencies — Neuron A learns to always correct Neuron B's mistakes. This is fragile. Dropout forces each neuron to be useful independently, learning more robust features.

Implementation detail: During training, multiply by mask and scale by 1/(1-p). During inference, no dropout (or equivalently, scale weights by (1-p)). This is called "inverted dropout" — keeps expected values consistent between train and inference.

Follow-up: "Why not use dropout in batch normalization layers or right before softmax?" - BatchNorm uses batch statistics that become noisy with dropout - Dropout after softmax would create non-probability outputs

Q6: What's the difference between batch normalization and layer normalization?

Model answer: Both normalize activations to stabilize training, but they differ in **which dimension** they normalize over:

BatchNorm: Normalizes across the batch dimension, per feature. - For a batch of B samples, each with N features: compute mean/var across B for each of the N features - Problem: behavior depends on batch size; different at train vs inference (uses running stats) - Great for: CNNs (images), large batch sizes

LayerNorm: Normalizes across the feature dimension, per sample. - For each sample independently: compute mean/var across its N features - Same behavior at train and inference - Works for any batch size (including batch_size=1) - Required for: Transformers, RNNs, variable-length sequences

Intuition: BatchNorm says "make this feature zero-mean unit-variance across the batch." LayerNorm says "make this sample's representation zero-mean unit-variance across features."

Q7: How do LSTMs solve the vanishing gradient problem in RNNs?

Model answer: Vanilla RNNs: `h_t = tanh(W_h * h_{t-1} + W_x * x_t)`. The gradient through time requires multiplying `dh_t/dh_{t-1} = W_h * tanh'(...)` for T steps → exponentially small.

LSTMs use an **additive cell state update:** `c_t = f_t * c_{t-1} + i_t * g_t`

The + (addition) is the key. The gradient of `c_t` w.r.t. `c_{t-1}` includes the forget gate `f_t`. When `f_t ≈ 1` (remember everything), the gradient flows unimpeded: `dc_t/dc_{t-1} = f_t ≈ 1`. This is the same "gradient highway" trick as ResNets.

The LSTM can learn to keep its forget gate open for long-range dependencies, allowing gradients to flow back hundreds of steps.

Follow-up: "Do LSTMs completely solve vanishing gradients?" No — they mitigate it significantly but don't eliminate it. For very long sequences (>500 steps), attention mechanisms (Transformers) work better because every position can directly attend to every other position.

Q8: Explain transfer learning and when fine-tuning is appropriate.

Model answer: Transfer learning uses knowledge from a pre-trained model (trained on a large dataset) as the starting point for a new task. Instead of random initialization, start from a model that already understands useful features.

When to use which strategy:

Your Data Size	Domain Similarity	Strategy
Small	Similar	Freeze all, train only head
Small	Different	Freeze early layers, fine-tune later layers + head
Large	Similar	Fine-tune entire network with small LR
Large	Different	Full fine-tuning or train from scratch

Practical tips: - Use learning rate ~10-100x smaller for pre-trained layers than new layers - Fine-tune in stages (unfreeze more layers progressively) - Warm up learning rate when unfreezing large pre-trained models

Why it works: Early CNN layers detect edges and textures that are universal across vision tasks. Early NLP layers learn syntax/semantics that transfer across text tasks. Only the final, task-specific layers need heavy adaptation.

SECTION 10

10 Senior-Level Discussion Points

1. The Double Descent Phenomenon

Classical statistics says: more model capacity → overfitting. Modern deep learning violates this. As model capacity grows beyond the interpolation threshold (where training loss = 0), test loss can *decrease again* — the "second descent." Large models that perfectly fit training data can still generalize well. This challenges bias-variance trade-off intuitions.

2. Neural Scaling Laws

Performance improves predictably as a power law with: number of parameters, dataset size, and compute. Kaplan et al. (OpenAI, 2020) showed these laws hold over many orders of magnitude. **Implication:** Given a compute budget, optimal allocation is a specific ratio of model size to data (Chinchilla scaling).

3. Loss Landscape Geometry

SGD with noise tends to find flatter minima (wider loss basins) than Adam. Flat minima generalize better — small perturbations to weights cause small changes in loss. Sharp minima are "brittle." This is one reason SGD sometimes generalizes better than Adam despite slower convergence.

4. Gradient Checkpointing

Trading compute for memory: instead of storing all activations during forward pass, recompute them during backward pass. Reduces memory from O(L) to O(√L) layers at the cost of ~33% more compute. Essential for training very deep models on limited GPU memory.

5. Mixed Precision Training

Use FP16 for forward/backward pass, FP32 for weight updates. 2x memory savings, ~3x throughput on modern GPUs. Critical: maintain FP32 master copy of weights, use loss scaling to prevent FP16 underflow of small gradients.

6. BatchNorm at Inference vs Training

BatchNorm uses batch statistics during training (noisy, slightly different each step) but uses running averages at inference (deterministic). This mismatch can cause subtle bugs — if your training pipeline has any distribution shift between batches, the running statistics diverge from batch statistics. Always call `model.eval()` before inference in PyTorch.

7. The Problem with Adam's Weight Decay

Standard Adam applies weight decay to the adaptive-LR-scaled gradient, not the weight directly. This means weight decay strength varies by parameter history. AdamW decouples weight decay: `W = W * (1 - wd) - lr * m_hat / (sqrt(v_hat) + ε)`. Always use AdamW for Transformers.

8. Dying ReLU in Practice

Networks with very large learning rates can have many neurons "die" in the first few training steps. Symptoms: training loss stops decreasing, gradient norms collapse. Solutions: reduce LR, use He initialization, use Leaky ReLU, use gradient clipping.

SECTION 11

11 Typical Mistakes Candidates Make

1. **"Backpropagation updates weights during the forward pass"** — No. Forward pass computes and stores activations. Backward pass computes gradients. Optimizer uses gradients to update weights. Three separate steps.
2. **"ReLU solves vanishing gradients completely"** — ReLU has gradient 1 for x>0, but dead neurons (gradient=0 for x<0) are another failure mode. Also, stacking many ReLU layers with no skip connections can still have issues.
3. **"Dropout is applied the same way at train and inference"** — No. Dropout is deactivated at inference time. In PyTorch: `model.eval()` turns off dropout; `model.train()` turns it back on.
4. **"BatchNorm removes the need for weight initialization"** — BatchNorm helps a lot but good initialization still matters, especially for the first few steps before BatchNorm statistics are meaningful.
5. **"Softmax in hidden layers"** — Softmax is for output layer only. Using softmax in hidden layers is almost always wrong (it creates competition between neurons where you want independence).
6. **"Larger learning rate always means faster training"** — Too large → divergence, oscillation, overshooting. Learning rate needs to be paired with batch size: if you double batch size, scale LR by $\sqrt{2}$ or 2 (linear scaling rule).
7. **"More layers always helps"** — Without residual connections, adding layers often hurts (degradation problem). Depth needs to be matched with appropriate architectural choices.
8. **"Adam always converges to the best solution"** — Adam converges faster but to different (sometimes worse generalizing) minima than SGD+momentum in some tasks. For production image classifiers (ResNets), SGD+momentum with careful LR schedule often wins.
9. **"Word2Vec gives the same embedding for 'bank' (river) and 'bank' (financial)"** — Static embeddings (Word2Vec) have one vector per word regardless of context. Contextual embeddings (ELMo, BERT) solve this.
10. **Confusing parameters vs hyperparameters** — Weights/biases are parameters (learned via gradient descent). Learning rate, batch size, number of layers, dropout rate, etc. are hyperparameters (set before training, tuned via validation).

SECTION 12

12 How This Connects to Other Topics

→ ML Fundamentals

- Gradient descent originated in classical ML (logistic regression, SVMs with SGD)
- Bias-variance trade-off, regularization (L1/L2), cross-validation: same concepts apply
- Neural networks are function approximators — they're just more powerful versions of linear models with non-linear feature transformations

→ Transformers / LLMs

- Self-attention is the core operation replacing RNN sequential processing
- LayerNorm is used instead of BatchNorm (batch-size independence)
- Embedding layers + positional encodings build directly on Word2Vec ideas
- Pre-training + fine-tuning transfer learning at massive scale
- GELU activation (vs ReLU) is standard in transformers

→ MLOps

- Training vs inference mode (BatchNorm, Dropout) must be handled correctly in serving
- Model serialization: saving/loading weights (`.pt`, `.safetensors`, ONNX)
- GPU memory management: gradient checkpointing, mixed precision
- Distributed training: data parallelism (split batch across GPUs), model parallelism (split model across GPUs)
- Experiment tracking: loss curves, gradient norms, activation distributions (TensorBoard, W&B)

→ Performance Engineering

- Operator fusion: combining Conv + BatchNorm + ReLU into single kernel
- Quantization: FP32 → INT8 for inference (4x memory, 2-4x throughput)
- Knowledge distillation: small model trained to mimic large model's soft outputs
- Pruning: remove unimportant weights (small magnitude), structured pruning for hardware efficiency
- Hardware: GPU tensor cores optimized for matrix multiplication, TPUs for large batch training

SECTION 13

13 FAANG Interview Tips

Coding questions (ML-adjacent): - Know how to implement forward propagation in numpy: `h = np.maximum(0, X @ W + b)` (ReLU + linear layer) - Know how to compute cross-entropy loss: `loss = -np.mean(np.log(y_hat[np.arange(n), y]))` - Know backprop for a simple 2-layer network — interviewers at Google/Meta sometimes ask this - Know how to implement a basic SGD update loop

ML design questions: - Always ask: how much data? Is it labeled? What are the compute constraints? What's the latency requirement? - For image tasks: start with pre-trained CNN backbone (EfficientNet/ResNet) + fine-tune - For text tasks: start with pre-trained BERT/RoBERTa + fine-tune - Always mention regularization, validation strategy, monitoring

Conceptual questions: - Have first-principles explanations ready, not just "it works because..." - Be able to derive the math: backprop chain rule, Xavier init formula, cross-entropy gradient - Connect to practical debugging: "how would you diagnose vanishing gradients?" → monitor gradient norms per layer, check activation distributions

Behavioral / ML judgment: - Know trade-offs: Adam vs SGD, CNN vs Transformer for vision, BatchNorm vs LayerNorm - Know failure modes: overfitting, underfitting, training instability, data leakage - Be ready to discuss real systems: recommendation, search ranking, content moderation

Common FAANG deep learning interview topics by company: - **Google:** Efficiency (EfficientNet, quantization, mobile models), distributed training, TensorFlow/Keras - **Meta:** Recommendation systems (DLRM, embeddings), PyTorch internals, ads models - **Amazon:** Time-series models (forecasting), on-device ML (Alexa), recommendation - **Apple:** On-device ML, CoreML, privacy-preserving training, efficient architectures

SECTION 14

14 Revision Cheat Sheet

10-Minute Summary

1. **Perceptron:** `y = f(Wx + b)`. Without `f`, it's linear. Non-linearity enables learning complex functions.
2. **MLP:** Stack layers. Each layer = linear transform + activation. Universal approximator with enough neurons.
3. **Forward prop:** Compute `z = Wx + b`, then `a = f(z)` for each layer. Store both.
4. **Loss:** MSE for regression. Cross-entropy for classification. Must be differentiable.
5. **Backprop:** Chain rule, layer by layer, right to left. `dL/dW = upstream * local_derivative * input`.
6. **Optimizers:** SGD (simple) → +Momentum (accelerate) → RMSprop (adapt LR) → Adam (both). AdamW for transformers.
7. **Vanishing gradients:** Sigmoid saturates → gradient dies. Fix: ReLU, residual connections, BatchNorm, He init.
8. **BatchNorm:** Normalize across batch per feature. Train and inference differ. Good for CNNs. **LayerNorm:** Normalize across features per sample. Same at train/inference. Good for Transformers/RNNs.
9. **Dropout:** Random mask during training, scale to compensate. Disabled at inference. Ensemble effect.
10. **CNNs:** Local, shared filters. Translation invariant. Feature maps = filter responses. Pool for downsampling.
11. **RNNs/LSTMs:** Sequential processing. Vanishing gradient through time. LSTMs: additive cell state = gradient highway. GRU = simpler LSTM.
12. **Transfer learning:** Pre-trained backbone + fine-tune head. Use smaller LR for pre-trained layers.

Key Numbers to Remember

Quantity	Value
Sigmoid derivative max	0.25 (at x=0)
Adam defaults	β1=0.9, β2=0.999, ε=1e-8
Xavier init variance	2/(n_in + n_out)
He init variance	2/n_in
Typical dropout rate	0.1 - 0.5
VGG-16 params	~138M
ResNet-50 params	~25M
LSTM: gates count	3 (input, forget, output)
GRU: gates count	2 (update, reset)
Softmax temperature	1.0 default; < 1 = sharper; > 1 = softer

Cheat-Sheet Table: Picking the Right Tool

Situation	Use
Image classification	CNN (ResNet, EfficientNet)
Sequence-to-sequence (NLP)	Transformer (BERT, GPT)
Time series / sequential	LSTM/GRU or Transformer
Small dataset, image task	Pre-trained CNN + freeze + new head
Binary output	Sigmoid + BCE loss
Multiclass output	Softmax + Cross-Entropy
Regression output	Linear + MSE
Hidden layers (default)	ReLU
Hidden layers (modern arch)	GELU
CNN training	BatchNorm
Transformer/RNN training	LayerNorm
Default optimizer	Adam (or AdamW for transformers)
Final accuracy push (vision)	SGD + momentum + cosine schedule
Prevent overfitting	Dropout, weight decay, data augmentation, early stopping
Training unstable	Gradient clipping, reduce LR, check init
Gradients vanishing	ReLU, residual connections, BatchNorm, He init

Most Important Concepts (Interview Priority)

P0 — Must know cold: - Forward propagation math - Backpropagation and chain rule - Why non-linearity matters - Vanishing gradient problem and solutions - Adam optimizer (what each term does) - BatchNorm vs LayerNorm - CNN: convolution, filter, stride, pooling, parameter count - LSTM gating mechanism (additive cell state) - Dropout mechanics (training vs inference) - Transfer learning strategy

P1 — Should know well: - Activation function comparison and trade-offs - Xavier vs He initialization - Residual connections (ResNet) - RNN vanishing gradient (through time) - Softmax + cross-entropy numerical stability - Word embeddings intuition - Overfitting diagnosis and fixes

P2 — Good to know for senior roles: - Double descent phenomenon - Adam vs SGD generalization trade-off - AdamW (decoupled weight decay) - Gradient checkpointing - Mixed precision training - GAN training dynamics - VAE latent space

This document covers ~95% of deep learning questions encountered in FAANG ML engineer and research scientist interviews. Review the Core Concepts section actively (write code, derive formulas) rather than passively reading.

ENGINEERING ESSENTIALS

Master the fundamentals.

Understand the why before the how. Connect every concept to a real system, an analogy, and a trade-off you can defend.

Vol. 20 — Deep Learning

FAANG Interview Master Series